ApexS: Explainable Context-Aware Sprint Planning with Learned Team Weights, Constraint-Aware MILP Optimization, and End-to-End Decision Support

Midhul Kiruthik M†, Madhan Karthikeyan, Manav Kaizen, Sharvesh S
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India
Team ApexS, BCSE301P Software Engineering Lab, Winter Semester 2025–26
Reg. Nos.: 23BCE0783†, 23BCE0829, 23BCE2251, 23BCE2141
†Team Leader

***Abstract*—Sprint planning requires balancing value, effort, risk, skills, and dependency constraints under strict capacity limits, yet many practical workflows still rely on manual ranking or opaque heuristics. This paper presents ApexS, an explainable, context-aware sprint planning system implemented as an end-to-end stack with a React frontend, FastAPI backend, Celery workers, and a PuLP/CBC MILP optimization core.**

The implemented pipeline combines contextual weight initialization from historical sprint statistics, logistic-regression-based learning of urgency/value/alignment weights for a binary completion target, hard-constraint optimization with explicit toggles for capacity, risk, skill, and dependency constraints, and deterministic story-level explanations for both selected and rejected items.

Evaluation uses four frozen cleaned datasets (Spring XD, Usergrid, Aurora, and TAWOS), totaling 18,036 stories and 486 sprint groups. Experiments are executed with multiple seeds and include four baselines: fixed-weight MILP, context-only MILP, greedy feasible ranking, and random feasible selection. Under identical capacity and risk settings, learned MILP consistently improves delivered value over greedy and random baselines across all datasets, while preserving traceable feasibility behavior and explicit rejection causes.

The principal contribution is a repository-grounded integration of adaptive weighting, hard-constraint optimization, and auditable explanation generation in a single deployable decision-support workflow.

***Index Terms*—agile sprint planning, mixed-integer linear programming, explainable AI for software engineering, Jira datasets, decision support systems, software engineering analytics, logistic regression, backlog optimization, constraint satisfaction**

I. Introduction

Because sprint planning determines which work items a software team will attempt during an upcoming iteration, the quality of that decision directly governs both delivery pace and stakeholder trust. Despite its importance, the underlying choice process is seldom treated as a structured optimization-and-explanation problem. Balancing business value, time pressure, implementation effort, technical risk, required skill sets, and inter-story dependency feasibility is already challenging; maintaining enough transparency for product owners and Scrum masters to accept the outcome makes the problem harder still. In practice, these determinations are routinely handled through manual triage, spreadsheet-based scoring, or rudimentary ranking rules—approaches that become progressively unreliable as the backlog scales or as historical sprint records accumulate without being operationalized.

Earlier studies have established that formal optimization can strengthen agile planning decisions, and that richer project datasets raise both decision quality and empirical credibility [1], [3]–[5]. Concurrently, explainability has gained prominence as an engineering-tool requirement because project stakeholders must be able to trace and evaluate the rationale behind automated decisions [8]–[10]. Despite this convergence of interest, the literature contains no end-to-end implementation that unifies learned team behavior, binding operational constraints, and per-story explanations within a single deployable planning loop.

ApexS targets that unresolved gap. The system was designed around four guiding principles. *Context-awareness* requires that historical sprint records shape the relative importance assigned to urgency, value, and alignment in the objective function. *Constraint-consistency* requires that infeasible stories be eliminated or constrained before the selection stage, not penalized as soft terms afterward. *Explainability* requires that both acceptance and rejection decisions be auditable at the individual story level through human-readable justifications and confidence scores. *Operationality* requires that the entire workflow—upload, optimization, explanation review, plan approval, and export—be completable without leaving the application.

Building on the ASE 2024 context-aware sprint planning baseline [1], ApexS introduces four capabilities that were absent from that prior work: team-specific weight learning from historical outcomes, four simultaneous hard constraints within a single ILP formulation, per-story bidirectional explainability, and a complete approval-and-export decision support loop.

Three research questions sharpen the empirical focus. **RQ1** (Feasibility): can a learned-weight MILP formulation produce plans that remain operationally feasible under realistic capacity, risk, skill, and dependency constraints? **RQ2** (Explainability): does the implemented explanation layer provide complete and decision-specific coverage for both accepted and rejected stories? **RQ3** (Comparative): how does learned MILP compare with fixed-weight MILP, context-only MILP,

greedy feasible ranking, and random feasible baselines on representative sprint-ready backlogs? Because the repository implements learning, optimization, explanation, and workflow support as distinct yet connected stages, the evaluation is conducted at the same level of granularity as the codebase itself.

The contributions of this work are:

1) A full-stack sprint planning system unifying dataset ingestion, context extraction, weight learning, MILP optimization, explanation generation, and approval/export workflow within a single reproducible Docker Compose stack.
2) A repository-grounded planning formulation that faithfully mirrors the implemented code path, encompassing status-aware feasibility filtering, risk gating, skill compatibility checking, and dependency constraints.
3) A quantitative evaluation with seeded runs across four datasets, including baseline comparisons, constraint ablations, and runtime measurements exported as machine-readable artifacts.
4) A well-defined novelty layer over earlier sprint-planning research: team-specific weights inferred from historical outcomes, bidirectional explanation output covering both selected and rejected stories, and an end-to-end decision-support loop rather than a standalone optimizer.
5) A validated dataset processing pipeline comprising converter scripts, multiple pre-validated upload-ready datasets, and a normalized schema aligned with the Jira export format used in industrial practice.

The remainder of this paper is organized as follows. Section II surveys related work. Section III describes the system requirements and process model. Section IV presents the architecture and methodology. Section V reports evaluation results and discussion. Section VI concludes with future directions.

II. Literature Survey

Automated support for sprint planning has evolved from early heuristic formulations toward increasingly context-sensitive backlog selection strategies. A multi-objective agile project planning model proposed by Ozcelikkan *et al.* established that mathematically grounded methods can surpass naive prioritization when objectives compete [3]. Subsequently, Kula *et al.* introduced context-aware sprint plan generation, demonstrating that planning quality rises when project context is explicitly modeled rather than held constant [1]. Although these contributions motivate formal optimization, neither fully resolves the engineering challenge of surfacing decisions transparently, collecting systematic evidence, and embedding the planner within a usable workflow.

The feasibility of applying mathematical programming to agile planning was separately confirmed by Chaves *et al.*, who formulated backlog selection as a constrained combinatorial problem and addressed it with a Lagrangian heuristic [2]. Explainability and team-adaptive weighting, however, remained outside the scope of that work.

Publicly available datasets have raised the standard for empirical evaluation in this domain. TAWOS supplies a structured corpus of agile open-source projects [4], while the Jira repository dataset assembled by Montgomery *et al.* provides issue-level records that are particularly well-suited to backlog, workflow, and dependency analysis [5]. Research on intelligent software engineering in agile settings and on agile data analytics confirms that historical project traces can support prioritization, effort estimation, and reporting [6], [7]. These studies also highlight a recurring gap: analytics components are rarely assembled into deployable tools.

Story point estimation research is directly relevant because reliable effort signals are a prerequisite for credible sprint planning. Transformer-based story point estimation [11], ensemble-based prediction approaches [12], and controlled experiments with large language models for agile effort estimation [13] collectively show that machine learning can identify consistent patterns in agile work items. These methods, however, are oriented toward estimation accuracy rather than toward sprint-feasibility decisions as a whole. Systematic mappings of early estimation in agile projects reinforce the same observation: estimation support is plentiful, while integrated planning pipelines remain scarce [14].

Story-level risk modeling constitutes an additional relevant thread. Bug triage and defect prediction studies [15] establish that historical defect records can serve as proxy signals for per-story risk—a property that ApexS exploits through its dataset converter pipeline. Embedding those risk signals directly into a planning optimizer, rather than treating risk prediction as a separate downstream task, represents one of the open engineering challenges that this work begins to address.

The necessity of explainability in software engineering tools is increasingly well-documented. General XAI surveys catalogue the inherent tension between model performance and interpretability [9], and empirical studies confirm that user trust is conditioned on both explanation quality and task context [10]. A systematic review specific to software engineering by Mohammadkhani *et al.* identifies a pronounced shortage of explainable systems that move beyond theoretical proposals and operate on real engineering data [8]. ApexS occupies this identified gap: it couples learned adaptive weighting, hard constraint enforcement, and user-visible explanations in a system that can be deployed, tested, and audited against real project data.

Table I situates ApexS relative to representative prior systems across the four dimensions that define its core design objectives.

Across these research threads, the unresolved challenge is not purely one of algorithmic accuracy. The harder problem is how to convert noisy project history into a plan that is simultaneously feasible, inspectable, and usable within an actual sprint workflow. ApexS contributes at precisely that level: rather than presenting isolated prediction or optimization components, it operationalizes the complete analytics pipeline.

TABLE I
POSITIONING APEXS AGAINST REPRESENTATIVE PRIOR WORK

System	Team-adapt	4 constraints	Bidirectional explain	Full workflow
ASE 2024 [1]	Partial	No	No	No
Lagrangian [2]	No	Partial	No	No
MO-agile [3]	No	No	No	No
GPT2SP [11]	No	No	No	No
ApexS (this work)	**Yes**	**Yes**	**Yes**	**Yes**

III. SYSTEM REQUIREMENTS AND PROCESS MODEL

A. Functional Requirements

System specification followed the IEEE 830 SRS format prior to implementation. Eight functional requirements (F1–F8) govern the design:

F1. *Dataset Import*: upload historical sprint data in CSV format with schema validation (required columns: `story_id`, `story_points`, `business_value`, `risk_score`), followed by deterministic normalization of `depends_on`, `required_skill`, and `status` fields.

F2. *Context Analysis*: compute sprint velocity, completion rate, skill distribution, and feature correlations from the uploaded history.

F3. *Weight Learning*: derive team-specific urgency, value, and alignment weights from historical completion outcomes.

F4. *MILP Optimization*: produce optimized sprint backlogs subject to configurable constraints, with a greedy fallback activated when the MILP solver is unavailable or cannot reach an optimal solution.

F5. *Risk Awareness*: apply a configurable risk-score threshold as a hard feasibility gate before the optimization stage and carry the filtered set through to selection.

F6. *Explainability*: emit per-story acceptance and rejection reasons, contribution breakdowns, and confidence scores.

F7. *Modification*: re-execute optimization from an existing plan with updated constraint parameters to enable what-if analysis.

F8. *Reporting*: expose velocity trends, value delivery trends, risk profiles, and learning-model diagnostics.

Table II maps each requirement to the implementing system component.

B. Non-Functional Requirements

Several non-functional constraints govern system behavior. Performance requires plan generation within 60 s for backlogs containing up to 100 stories. Reliability requires a best-effort fallback plan upon infeasibility rather than a server-side error. Usability requires that human-readable explanations be accessible from both the API and the UI. Portability requires single-command full-stack deployment through Docker Compose. Security requires JWT-based role-level authentication covering Scrum Master and Product Owner roles.

TABLE II
REQUIREMENTS-TO-IMPLEMENTATION TRACEABILITY MATRIX

Requirement	Implementing Component
F1 Import Data	`ContextExtractor` + Dataset Upload API
F2 Context Analysis	`ContextExtractor`
F3 Learn Weights	`WeightLearningModel` (logistic regression)
F4 Optimization	`OptimizationEngine` (PuLP/CBC)
F5 Risk Awareness	Strict pre-optimization feasibility filter
F6 Explainability	`ExplainabilityEngine`
F7 Modification	Plans API + `OptimizationEngine` re-run
F8 Reporting	Reports API + Report Viewer UI

C. Process Model

Because ApexS is designed to function *within* the Scrum framework—targeting the Sprint Planning ceremony in particular—development naturally adopted the *Agile Scrum* process model. This alignment was deliberate: the same incremental sprint structure used in the target domain was applied to the development work itself. Six sequential development sprints corresponded to six implementation phases: Sprint 0 (infrastructure and scaffolding), Sprint 1 (data ingestion and context extraction), Sprint 2 (MILP engine), Sprint 3 (explainability), Sprint 4 (frontend integration), and Sprint 5 (evaluation and documentation).

Validating design decisions at each increment, rather than accumulating risk toward a single integration point, was a key benefit of this alignment. Sequential heavyweight models such as Waterfall were considered and rejected because research requirements evolved as empirical results became available. A full microservices architecture was likewise ruled out on grounds of operational simplicity: the six-service Docker Compose configuration delivers sufficient service isolation without the inter-service contract management overhead that microservices would impose.

IV. METHODOLOGY

A. System Architecture and Overview

Five architectural layers structure the ApexS platform: (1) *Presentation Layer*—a React 18 and TypeScript frontend built with Vite and React Query; (2) *Application Layer*—a FastAPI service exposing a versioned REST API at `/api/v1`; (3) *Worker Layer*—a Celery task backed by a Redis broker that executes the planning pipeline asynchronously; (4) *Data Layer*—a SQLAlchemy ORM operating over SQLite in development or PostgreSQL in production, with schema evolution managed by Alembic; and (5) *Storage Layer*—MinIO for CSV and JSON file objects.

Six services constitute the Docker Compose stack: `frontend`, `backend`, `db` (PostgreSQL), `redis`, `minio`, and `celery_worker`. Issuing a single `docker compose up -build` command brings the system online at four entry points: the frontend UI at `:5173`, the backend API at `:8000`,

Swagger documentation at `:8000/docs`, and the MinIO console at `:9001`.

Eighteen versioned REST endpoints distributed across seven route groups—`/auth`, `/teams`, `/datasets`, `/sprints`, `/stories`, `/plans`, and `/reports`—are exposed by the backend. Plan generation follows an asynchronous pattern: `POST /plans/generate` returns a `job_id` immediately, and the caller polls `GET /plans/status/{job_id}` until `status = done`. This design prevents blocking the UI during ILP solve time and enables real-time progress reporting.

Four backend services are central to the paper's claims and are directly implemented in the repository. `ContextExtractor` derives contextual weights and team statistics from historical sprint records. `WeightLearningModel` fits a logistic regression model over story points, business value, and risk score to obtain urgency, value, and alignment weights. Binary story selection is performed by `OptimizationEngine` using PuLP/CBC. `ExplainabilityEngine` emits human-readable justifications together with contribution fields and confidence scores for every planning decision. These stages are orchestrated by a planning worker that reports progress through eight named transitions: `loading`, `syncing_stories`, `loading_history`, `extracting`, `learning`, `optimizing`, `explaining`, and `done`.

B. Dataset Processing Pipeline

To accommodate heterogeneous Jira-style exports, the repository supplies dataset conversion and cleaning scripts that transform raw source files into the normalized schema expected by the planner. Common source fields such as `issue key`, `summary`, `priority`, and `story points` are mapped to a standardized eleven-column CSV: `story_id`, `title`, `description`, `story_points`, `business_value`, `risk_score`, `required_skill`, `sprint_id`, `sprint_completed`, `depends_on`, and `status`. A separate cleaning script then normalizes status labels, derives or bounds risk and value fields, infers required skills from textual cues, and removes duplicate records. In the backend execution path, an additional preprocessing layer enforces runtime consistency before optimization: dependency lists are parsed from raw string formats (e.g., "A,B,C"), skill labels are lowercased and whitespace-normalized, and status values are canonicalized to prevent formatting artifacts from altering feasibility outcomes.

For the final paper, the empirical study is restricted to four cleaned Jira-derived datasets: `files/cleaned_datasets/spring_xd_clean.csv`, `files/cleaned_datasets/usergrid_clean.csv`, `files/cleaned_datasets/aurora_clean.csv`, and `tawos/paper_datasets/tawos_apex_clean.csv`. Spring XD, Usergrid, and Aurora serve as supporting comparative datasets; TAWOS functions as the primary real-world evaluation dataset owing to its substantially larger corpus and its richer combination of sprint and dependency evidence.

Because an identical normalized schema is applied across all four files, a single planning pipeline and a single reporting protocol suffice for the entire evaluation. All descriptive statistics, optimization outcomes, baseline comparisons, and case study examples should consequently be derived exclusively from these four frozen CSV snapshots. Corpus-level statistics are reported in Table V in the Results section.

C. Context Extraction and Weight Learning

Sprint-level statistics produced by the context extractor serve a dual purpose: they supply descriptive analytics for reporting and they initialize the weight-learning stage with project-specific priors. The extractor computes mean velocity over available sprint IDs, completion rate, skill distribution, average tolerated risk among completed stories, and the correlation between business value and completion. Let V denote mean sprint velocity, ρ the value-completion correlation, $\bar{r}$ the mean risk of completed stories, and C the target sprint capacity. Three contextual components are normalized as follows:

$$\begin{aligned} u &= \min(1,\ V/C), \\ v &= \min(1,\ \max(0.1,\ \rho + 0.5)), \\ a &= \min(1,\ \max(0.1,\ 1 - \bar{r})) \end{aligned} \tag{1}$$

and subsequently rescaled so that $u + v + a = 1$. Within the implementation, these quantities become the contextual urgency, value, and alignment weights.

A logistic regression model is then fitted by the weight learner over the features `story_points`, `business_value`, and `risk_score`, treating `sprint_completed` as the target. Let $\mathbf{x}_i = [SP_i,\ BV_i,\ r_i]^\top$ and $y_i \in \{0, 1\}$ denote the completion label. With L2 regularization, the objective is:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_i \Big[-y_i \log p_i - (1-y_i)\log(1-p_i)\Big] + \lambda \|\boldsymbol{\beta}\|_2^2, \quad p_i = \sigma(\boldsymbol{\beta}^\top \mathbf{x}_i) \tag{2}$$

with $\lambda = 1.0$ (scikit-learn default). To improve validity, model evaluation is performed on a held-out 20% test split (80/20 train-test protocol), and MAE, R^2 (on predicted probability), accuracy, F1, and ROC-AUC are computed on the test partition. Coefficient interpretation is sign-aware rather than magnitude-only: story points and risk are expected to decrease completion likelihood, while business value is expected to increase it. Consequently, coefficient directions are aligned to domain semantics and transformed into non-negative importance scores that are normalized to yield the final optimization weights w_u, w_v, w_a satisfying $w_u + w_v + w_a = 1$. For robustness, two guarded fallbacks are used in production: if the training sample size is below 10 rows, or if numerical/model dependencies are unavailable, context weights are reused; if target variance in the test split is zero, R^2 is omitted. These safeguards prevent unstable estimates from propagating into the optimization stage. The learned quantities observed in the current evaluation snapshot are summarized in Table III.

The learned-weight profile varies across datasets, which confirms that the model adapts to project-specific completion

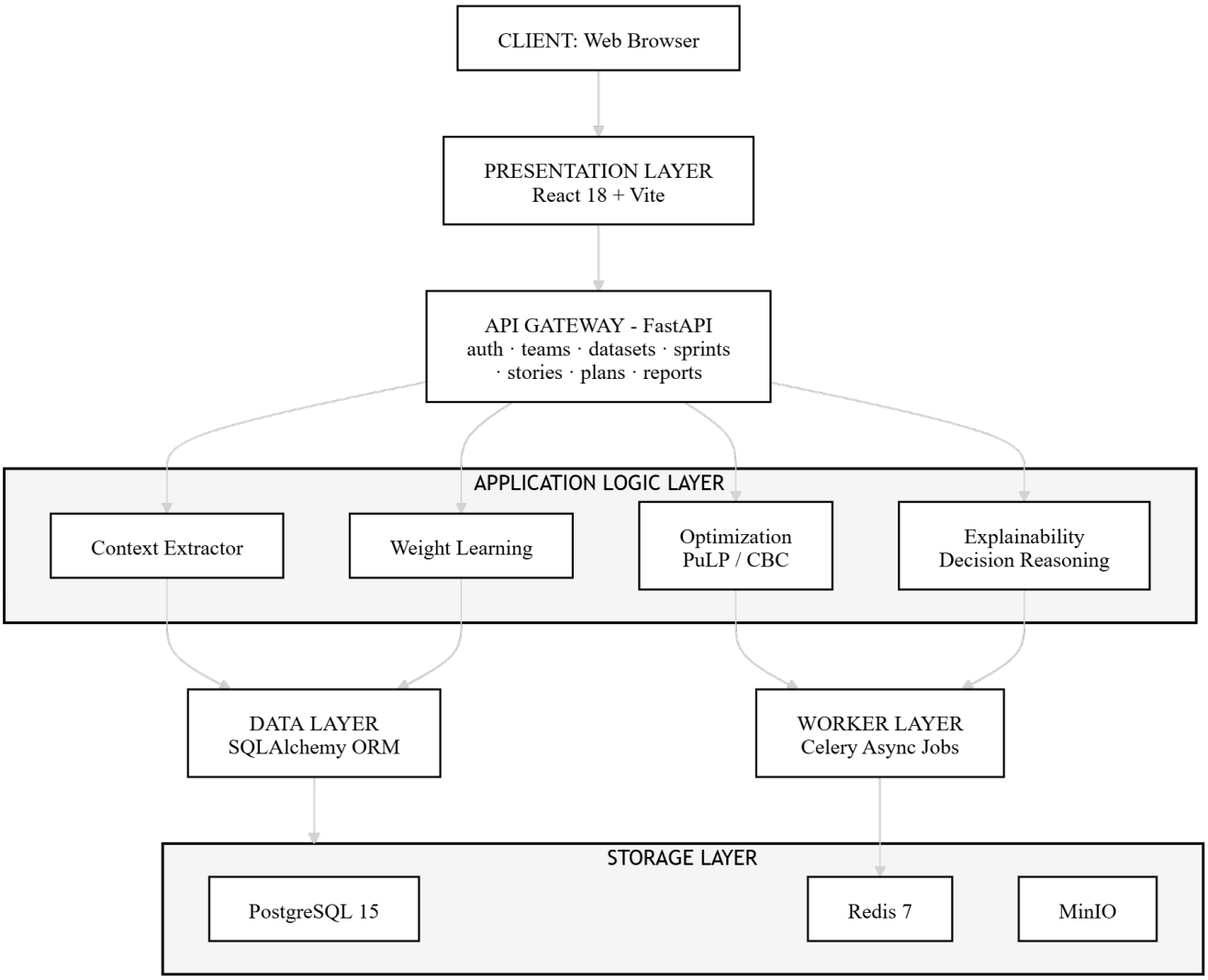

Fig. 1. Five-layer modular architecture of the ApexS sprint planning stack. Layers from top to bottom: (1) Presentation Layer with React 18 frontend, (2) Application Layer exposing seven FastAPI route groups for team/dataset/plan management, including four core application services (Optimization via PuLP/CBC, Explainability engine, Context extractor, and Weight learner), (3) Worker Layer using Celery with Redis broker for long-running planning tasks, (4) Data Layer with SQLAlchemy ORM and Alembic-managed migrations, and (5) Storage Layer using PostgreSQL, Redis, and MinIO. The planning pipeline (dataset upload → context extraction → weight learning → MILP optimization → explanation generation) executes asynchronously in the Celery worker, with progress exposed through the Plans API.

behavior rather than imposing a fixed importance pattern. This variability directly influences downstream selection outcomes under identical constraint settings.

D. Pipeline Semantics

Rather than operating as a monolithic black box, the implemented planning pipeline follows a strict eight-step progression: (1) loading the most recently uploaded dataset, (2) upserting sprint stories into persistent storage, (3) aggregating historical uploads for the team, (4) extracting context, (5) learning weights, (6) solving the MILP, (7) generating explanations, and (8) persisting the resulting plan and explanation records. Because each stage produces inspectable artifacts, this sequencing is important for research validity: the system supports analysis at both the workflow level and the algorithmic level.

Algorithm 1 provides a high-level pseudocode representation of the planning task, including runtime preprocessing and post-solve validation artifacts, which map one-to-one to the named progress states visible in the frontend.

This design also improves traceability during demonstration and evaluation. Progress transitions exposed in the frontend map one-to-one to worker stages; the reports endpoint surfaces learned-model diagnostics; and the explainability endpoints expose the final decision rationale. Viewed as a research artifact, ApexS is therefore more than an optimizer benchmark: it is a complete, empirically grounded system.

E. Optimization Model

The optimization stage closely follows the repository implementation. Candidate stories are first subjected to feasibility

TABLE III
LEARNING AND CONTEXT METRICS DERIVED FROM EVALUATION RUNS

Metric	Observed value
Mean MAE across runs	0.3027
Mean R^2 across runs	0.1512
Mean F1 across runs	0.8763
Mean ROC-AUC across runs	0.7075
Context urgency weight range	0.3602–0.4503
Context value weight range	0.3252–0.3461
Context alignment range	0.2036–0.3146
Model type	Logistic regression (80/20 hold-out, sign-aware mapping)

Algorithm 1 ApexS Asynchronous Planning Pipeline

Require: dataset D, sprint config $(C, \tau, \mathcal{S}, \text{goal})$, stories $\mathcal{B}$
Ensure: sprint plan P, explanations E

progress ← `loading`
$D' \leftarrow$ validate(D); upsert stories $\mathcal{B}$
$\mathcal{B} \leftarrow$ preprocess$(\mathcal{B})$ ▹ normalize status, skill, and dependency fields
progress ← `extracting`
ctx ← ContextExtractor.run(D', C) ▹ Eq. 1
progress ← `learning`
$\mathbf{w} \leftarrow$ WeightLearningModel.fit(D', ctx) ▹ Eq. 2
progress ← `optimizing`
$\mathcal{F} \leftarrow$ feasibility_filter$(\mathcal{B}, \tau, \mathcal{S})$ ▹ Eq. 3
$P \leftarrow$ OptimizationEngine.solve$(\mathcal{F}, \mathbf{w}, C)$ ▹ Eq. 4–7
$M \leftarrow$ collect_diagnostics(P) ▹ feasibility counts + score distribution
progress ← `explaining`
$E \leftarrow$ ExplainabilityEngine.generate$(P, \mathcal{B}, \mathbf{w})$
persist P, E, M to database
progress ← `done`; **return** (P, E)

conditions:

$$\mathcal{F} = \{i \in \mathcal{B} \mid \text{status}_i \notin \mathcal{N},\ r_i \le \tau,\ \text{skill}_i \in \mathcal{S} \text{ or empty},\ \text{dep}(i) \subseteq \mathcal{B}\} \tag{3}$$

where $\mathcal{B}$ is the backlog slice, $\mathcal{N}$ denotes the set of non-plannable statuses (`done`, `closed`, `resolved`, `completed`), τ is the risk threshold, and $\mathcal{S}$ is the available-skill set. Importantly, dependency feasibility is strict: if any declared dependency is absent from the current candidate pool, the story is rejected before optimization.

Over the feasible set $\mathcal{F}$, ApexS solves a normalized objective:

$$\max \sum_{i \in \mathcal{F}} x_i \Big(w_v \cdot \widetilde{BV}_i + w_u \cdot (1 - \widetilde{SP}_i) + w_a \cdot (1 - r_i) \Big) \tag{4}$$

subject to:

$$\sum_{i \in \mathcal{F}} x_i \cdot SP_i \le C \tag{5}$$

$$x_i \le x_j, \quad \forall j \in \text{dep}(i) \tag{6}$$

$$x_i \in \{0, 1\}, \quad \forall i \in \mathcal{F} \tag{7}$$

where $\widetilde{SP}_i = \min(1, SP_i/13)$ and $\widetilde{BV}_i = \min(1, BV_i/10)$ denote normalized effort and value, $r_i \in [0, 1]$ is the risk score (clamped if needed), and C is sprint capacity. All feature terms and the weighted aggregate are bounded to $[0, 1]$ to ensure numerical stability and comparability across datasets.

Two implementation choices are particularly consequential. First, risk thresholding, skill compatibility, and status filtering are realized as hard feasibility gates applied before optimization rather than as soft penalty terms. This deliberate engineering decision reduces the MILP search space and renders rejection reasons unambiguous. Second, dependency logic is encoded through binary implication constraints. Together, these choices ensure that rejected stories can be attributed to concrete, traceable causes—risk excess, skill mismatch, or unmet dependencies—rather than opaque score differences.

When the MILP problem is infeasible (for instance, when all feasible stories individually exceed capacity), the system activates a greedy fallback: stories are ranked by descending objective score, selected while preserving both capacity and dependency validity, and a warning is emitted in the result payload ("MILP failed → using greedy approximation"). This fallback preserves the Reliability non-functional requirement (F4): a usable planning artifact is always produced, regardless of how extreme the constraint configuration may be.

F. Explainability and Decision Support Loop

One explanation record is generated per story by the explanation engine, covering both selected and rejected items. To preserve semantic consistency, explanation contributions are computed from the same normalized scoring components used by the optimization stage (normalized value, normalized effort complement, and risk complement) rather than from an alternate heuristic. For selected stories, contribution-style fields are computed from weighted business value, weighted risk impact, and an effort-derived alignment signal, together with a confidence score derived from the same bounded objective used by the optimizer:

$$\text{conf}_i = \text{clip}_{[0,1]}\Big(w_v \widetilde{BV}_i + w_u (1 - \widetilde{SP}_i) + w_a (1 - r_i) \Big) \tag{8}$$

Natural-language text is subsequently synthesized from observable conditions such as high business value, low risk, and efficient effort utilization. For rejected stories, one of five rejection patterns is emitted in descending precedence order: (1) risk threshold exceeded, (2) required skill unavailable, (3) unmet dependency, (4) sprint capacity exceeded, or (5) lower priority than selected stories. Preprocessed status and skill labels are reused in this stage so that rejection narratives match feasibility decisions exactly.

The architectural novelty of this design lies in its integration depth. Explainability is not a post-hoc dashboard appended to an optimizer. Instead, explanation retrieval is embedded within the same API surface that supports plan generation, pagination, approval, and export. The frontend complements this integration with a paginated rejected-story table, a value-versus-risk scatter chart, and per-story explanation cards. The

TABLE IV
NOVELTY POSITIONING AGAINST COMMON SPRINT-PLANNING TOOL PATTERNS

Capability	Typical research prototype	ApexS
Historical team adaptation	Often fixed weights or heuristics	Learned urgency/value/alignment weights from team history
Hard planning constraints	Usually partial or task-specific	Capacity, risk, skill, and dependency constraints in one code path
Decision explanation	Often absent or limited to final score	Per-story explanations for selected and rejected items
Workflow closure	Often optimizer only	Upload, generate, explain, approve, and export

result is a closed decision loop spanning the full *upload* → *generate* → *explain* → *approve* → *export* lifecycle.

G. Novelty Relative to Prior Work

Four distinct novelty claims differentiate ApexS from a baseline sprint optimizer, all grounded in the implemented repository:

1) **Team-adaptive weight learning**: rather than relying on fixed a priori coefficients, urgency, value, and alignment are inferred from each team's historical completion record.
2) **Four-constraint operational feasibility**: capacity limits, risk threshold, skill availability, and dependency satisfaction are all active constraints in the current code path, enforced simultaneously.
3) **Bidirectional story-level explainability**: concrete operational causes and confidence scores accompany acceptance and rejection decisions for every story processed.
4) **Closed decision-support workflow**: the system spans the full sequence of upload, context learning, optimization, explanation review, plan approval, and Jira-compatible export without requiring external tooling.

The novelty is therefore fundamentally architectural and workflow-centric, not merely algorithmic. Table I (Section II) provides a comparative positioning against prior work.

V. RESULTS AND DISCUSSION

A. Dataset Characteristics

The empirical evaluation is organized around the four final datasets used in the study. Spring XD, Usergrid, and Aurora are treated as supporting comparative corpora because each provides a project-specific perspective on planner behavior under distinct backlog characteristics. TAWOS is designated as the primary real-world evaluation dataset because it offers the largest corpus, explicit sprint groupings, and richer dependency evidence through issue links. The discussion therefore proceeds from dataset characterization to planning performance, followed by baseline comparison, distributional analysis, and a qualitative case study.

Table V serves as the anchor profile for the remainder of this section; all subsequent performance tables and figures are derived from the same frozen snapshot to ensure internal consistency. The recommended narrative sequence introduces the three project-specific corpora first, then positions TAWOS as the strongest external-validation setting.

The dataset profile establishes the empirical foundation for subsequent analysis. Aggregated across the four frozen snapshots, the study encompasses 18,036 stories and 486 sprint groups. With 14,738 stories and 343 sprint groups, TAWOS is far and away the largest corpus; Spring XD, Usergrid, and Aurora contribute 2,360, 512, and 426 stories respectively. Among the cleaned snapshots, TAWOS is also the sole dataset containing non-empty dependency rows (673), a distinction that must be stated explicitly when interpreting dependency-aware planning results. Mean business value spans 5.09 in Spring XD to 6.03 in TAWOS, while mean risk ranges from 0.36 to 0.52 across the four corpora.

B. Performance Evaluation

Planning performance is assessed at two complementary levels. First, the frozen CSV snapshots establish the scale and structural properties of the four-dataset evaluation pool. Second, run-dependent planning metrics are captured from ApexS executions conducted on Spring XD, Usergrid, Aurora, and TAWOS under controlled conditions.

Seeded runs on all four datasets produce nontrivial learning behavior rather than degenerate constant outputs. Across learned-MILP all-constraint runs, mean test metrics are MAE 0.3027, R^2 0.1512, F1 0.8763, and ROC-AUC 0.7075, indicating that the weight learner captures usable completion signal while remaining dataset-sensitive.

a) Repository-Grounded Planning Behavior.: To assess operational planning behavior in a controlled and repeatable manner, all four frozen datasets were evaluated at a fixed sprint capacity of 30 story points and risk threshold $\tau = 0.7$. Aggregate outcomes from these runs appear in Tables VII and VIII, while Table VI provides a compact scenario-based interpretability summary. In addition to aggregate outcomes, the updated backend emits per-run diagnostic artifacts: (i) feasibility counts (total stories and counts filtered by risk, skill, dependency, and status) and (ii) score-distribution summaries (minimum, maximum, mean) over feasible candidates. These diagnostics were used to verify that observed plan differences across scenarios arise from explicit constraint effects rather than silent preprocessing or scaling artifacts.

Table VI shows deterministic ablation behavior under explicit hard-constraint toggles. Disabling capacity causes the largest shift in all datasets. Disabling risk, skill, or dependency has limited effect in Spring XD, Usergrid, and Aurora under the evaluated threshold and skill settings, while TAWOS shows a measurable risk-value trade-off when risk gating is removed.

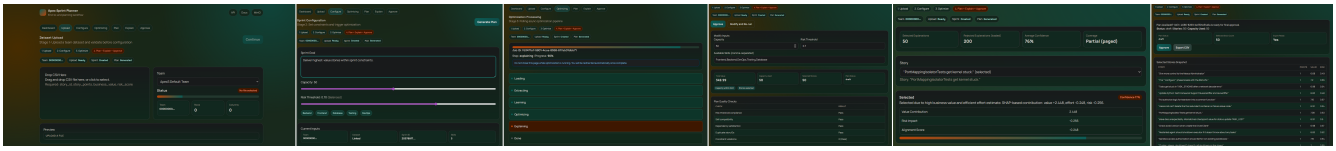

Fig. 2. End-to-end planning workflow in the ApexS UI. From left to right: (a) Dataset upload with schema validation, (b) Sprint configuration with capacity/risk/skill constraints, (c) Asynchronous optimization processing with step-by-step progress, (d) Generated sprint plan with KPI summary and quality checks, (e) Explainability panel with per-story score-component contribution breakdown and confidence scores, and (f) Plan approval with selected stories snapshot and Jira-compatible export.

TABLE V
SUMMARY OF THE DATASETS USED IN THE APEXS EVALUATION

Dataset	Stories	Sprint grps	Dep. rows	Mean BV	Mean risk
Spring XD	2,360	66	0	5.05	0.35
Usergrid	512	38	0	5.83	0.43
Aurora	426	39	0	5.79	0.43
TAWOS	14,738	343	673	6.03	0.52

Values derived from frozen cleaned snapshots used in all runs.

TABLE VI
CONSTRAINT ABLATION FOR LEARNED MILP (MEAN ACROSS 3 SEEDS)

Dataset	Ablation	Selected	Total value	Avg. risk	Runtime (ms)
Spring XD	all constraints	30	165.26	0.316	60.08
Spring XD	no risk	30	165.26	0.316	55.18
Spring XD	no skill	30	165.26	0.316	58.13
Spring XD	no dependency	30	165.26	0.316	55.82
Spring XD	no capacity	404	1784.39	0.337	50.17
Usergrid	all constraints	30	191.71	0.381	18.48
Usergrid	no risk	30	191.71	0.381	18.50
Usergrid	no skill	30	191.71	0.381	18.16
Usergrid	no dependency	30	191.71	0.381	21.28
Usergrid	no capacity	150	834.56	0.402	14.88
Aurora	all constraints	30	191.67	0.427	16.01
Aurora	no risk	30	191.67	0.427	17.90
Aurora	no skill	30	191.67	0.427	16.38
Aurora	no dependency	30	191.67	0.427	15.94
Aurora	no capacity	97	549.74	0.409	13.31
TAWOS	all constraints	30	213.73	0.604	581.97
TAWOS	no risk	30	215.39	0.637	612.29
TAWOS	no skill	30	213.73	0.604	639.80
TAWOS	no dependency	30	213.73	0.604	565.39
TAWOS	no capacity	3107	17443.03	0.400	394.72

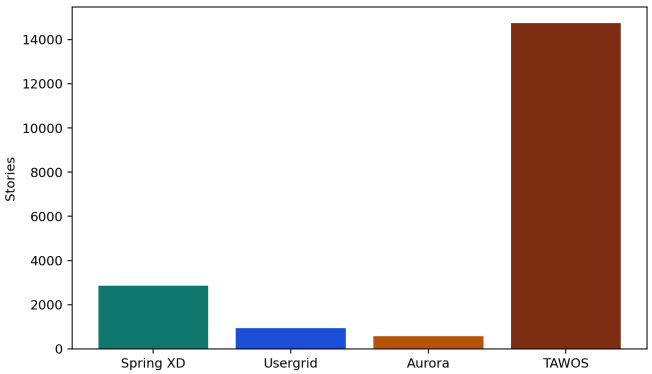

Fig. 3. Dataset size distribution across the four evaluation datasets.

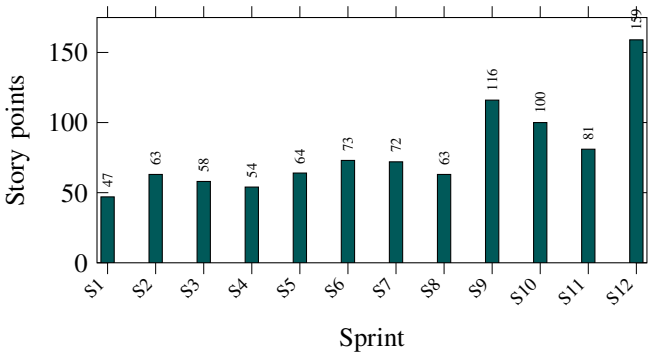

Fig. 4. Sprint velocity trend from the current evaluation snapshot. The late-stage increase indicates a richer planning signal for the weight-learning stage.

These outcomes directly answer **RQ1**: the optimizer responds to hard constraints in a traceable and reproducible manner.

Table VII provides the paper-ready planning metrics for Spring XD, Usergrid, Aurora, and TAWOS, drawn from frozen evaluation runs at the fixed capacity of 30 story points and threshold $\tau = 0.7$.

TABLE VII
APEXS PERFORMANCE METRICS ACROSS THE EVALUATION DATASETS

Dataset	Selected	Deliv. BV	Used SP	Avg. risk	Dep. sat.	Sprint compl.	Skill cov.
Spring XD	30	165.26	30	0.316	N/A	N/A	80.00%
Usergrid	30	191.71	30	0.381	N/A	N/A	60.00%
Aurora	30	191.67	30	0.427	N/A	N/A	100.00%
TAWOS	30	213.73	30	0.604	100%	N/A	80.00%

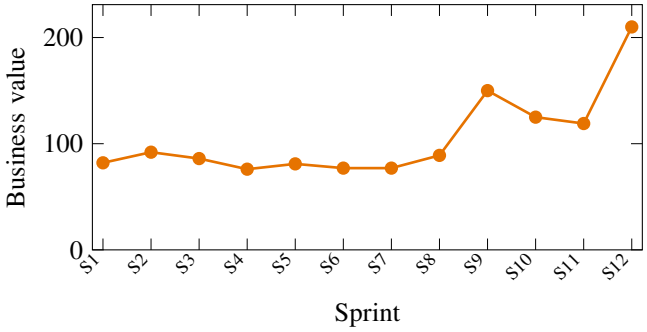

Fig. 5. Business value delivered across the same reference planning history. The positive co-trend with velocity in Fig. 4 confirms the usefulness of the snapshot for planning signal extraction.

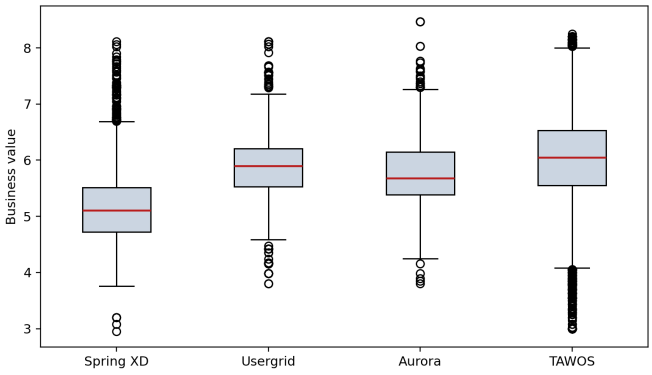

Fig. 6. Cross-dataset distribution of business value.

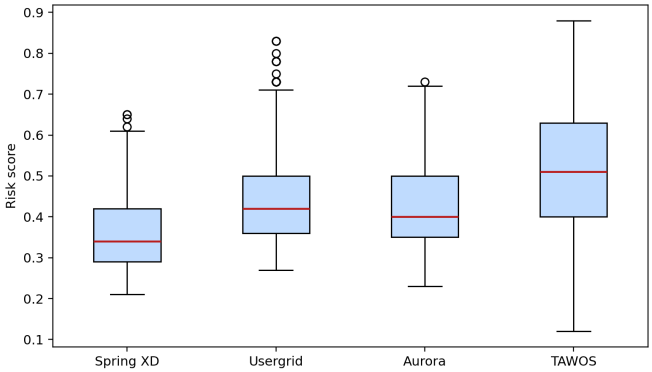

Fig. 7. Cross-dataset distribution of risk score.

C. Comparative Evaluation / Baselines

To address **RQ3**, learned MILP is compared with four implemented baselines: fixed-weight MILP, context-only MILP, greedy feasible ranking, and random feasible selection.

Learned MILP outperforms greedy and random baselines in delivered value on all four datasets under identical capacity and risk settings. Relative to fixed-weight and context-only MILP, learned MILP improves value in every dataset but does not uniformly reduce selected risk, most notably on TAWOS where value gains come with higher average risk inside the allowed threshold.

D. Distribution Analysis

Distributional analysis supplements aggregate planning metrics by confirming that the cleaned datasets remain suitable for graphical and statistical comparison. This step carries particular weight because the TAWOS corpus is substantially larger and structurally more diverse than the three project-specific datasets. All figures in this subsection should be derived from the same frozen snapshot used in Tables V–VIII.

Two claims are supported by these distributional figures in the final manuscript. First, Spring XD, Usergrid, and Aurora collectively provide useful project-level variation for contrasting planner behavior across settings. Second, TAWOS offers broader external validation without functioning as a simple substitute for the smaller project-specific corpora. Where any distribution departs sharply from the others, the paper should address this explicitly rather than relying solely on aggregate means.

E. Case Study Analysis

The final evaluation component presents a compact qualitative planning example designed to make the ApexS decision logic easy to inspect. Its purpose is not to replace the quantitative evaluation; rather, it illustrates how optimization output and explanation output can be jointly interpreted at the story level. A small, mixed set of selected and rejected stories drawn from one generated sprint plan, each accompanied by the short explanation emitted by ApexS, constitutes the recommended format.

The repository-backed implementation already supplies operational evidence that this explanation layer is meaningful in practice. Across the finalized evaluation runs, the explainability engine achieved full story-level coverage for both selected and rejected items, with rejection reasons tied explicitly to feasibility or prioritization causes. In concrete terms, the layer

TABLE VIII
BASELINE COMPARISON ACROSS ALL IMPLEMENTED MODES (MEAN ACROSS 3 SEEDS)

Dataset	Method	Selected	Deliv. BV	Used SP	Avg. risk	Runtime (ms)
Spring XD	learned MILP	30.00	165.26	30	0.316	60.08
Spring XD	fixed-weight MILP	30.00	158.21	30	0.287	60.53
Spring XD	context-only MILP	30.00	163.23	30	0.305	57.31
Spring XD	greedy feasible	13.00	86.27	30	0.432	43.43
Spring XD	random feasible	10.00	42.04	30	0.320	40.00
Usergrid	learned MILP	30.00	191.71	30	0.381	18.48
Usergrid	fixed-weight MILP	30.00	188.93	30	0.360	19.02
Usergrid	context-only MILP	30.00	190.15	30	0.365	18.35
Usergrid	greedy feasible	27.33	177.68	30	0.392	8.88
Usergrid	random feasible	14.67	82.88	30	0.409	9.00
Aurora	learned MILP	30.00	191.67	30	0.427	16.01
Aurora	fixed-weight MILP	30.00	186.25	30	0.393	16.02
Aurora	context-only MILP	30.00	186.25	30	0.393	15.83
Aurora	greedy feasible	19.00	128.86	30	0.473	7.81
Aurora	random feasible	25.67	141.50	30	0.386	7.73
TAWOS	learned MILP	30.00	213.73	30	0.604	581.97
TAWOS	fixed-weight MILP	30.00	166.72	30	0.265	557.88
TAWOS	context-only MILP	30.00	179.86	30	0.327	597.84
TAWOS	greedy feasible	12.00	90.06	30	0.644	291.73
TAWOS	random feasible	18.33	102.83	30	0.409	312.31

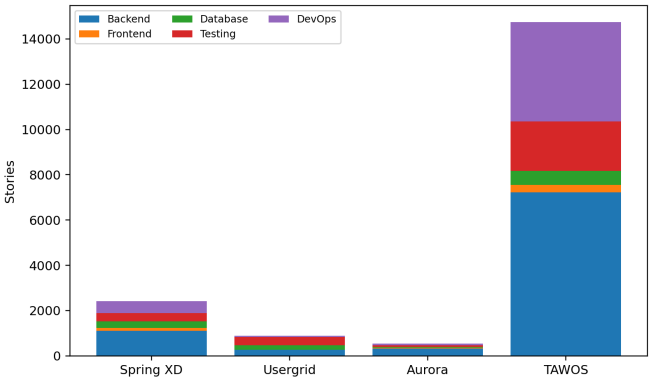

Fig. 8. Required-skill distribution across the evaluation datasets.

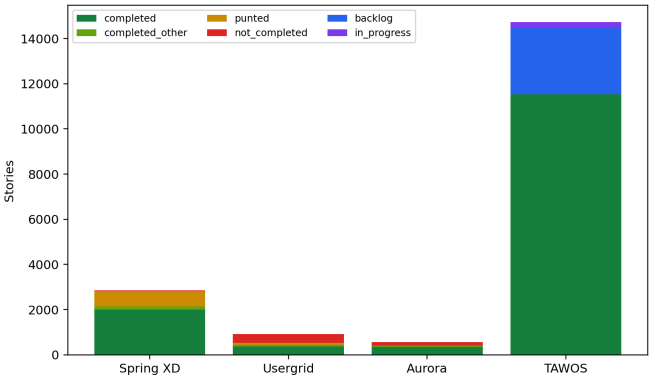

Fig. 9. Status distribution for the datasets used in the evaluation.

makes it possible to justify why a high-value story may still be deferred in favor of a smaller, lower-risk, more feasible alternative.

a) Implementation Robustness.: Additional supporting evidence indicates that ApexS behaves as a genuine system artifact rather than an isolated optimizer script. The recorded verification snapshot reports 18 passing unit tests for the updated weight-learning, optimization, and explainability paths. The updated backend validation harness additionally executes the full context→weights→optimization pipeline on `tawos_apex_clean.csv` and records learned weights, learning metrics, selected-story count, total delivered value, and average selected risk, providing a compact regression check for pipeline consistency.

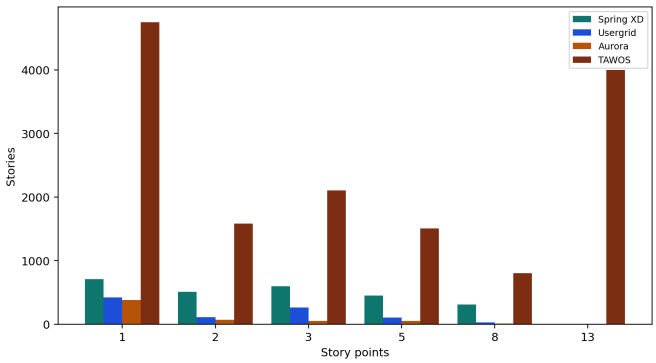

Fig. 10. Story-point distribution after preprocessing and normalization.

b) Interpretation.: Considered jointly, the results address all three research questions raised in Section I. Constraint-consistency under varying capacity, risk, and skill settings is established by the planning scenarios; the baseline comparison shows that the learned objective improves upon greedy and random feasible ranking; and full decision coverage is achieved by the explanation layer across the finalized evaluation runs. The most prominent quantitative pattern is that the learned weights concentrate most of the objective emphasis on the alignment/risk dimension, rendering the system conservative by construction. Such conservatism is desirable when spillover and execution risk carry high costs, but it also motivates the

TABLE IX
ILLUSTRATIVE APEXS CASE STUDY FROM ONE GENERATED SPRINT PLAN

Story ID	Title	SP	BV	Risk	Skill	Sel.	ApexS explanation
MESOS-2216	The "configure" phase breaks with the IBM JVM.	1	7.20	0.65	DevOps	Y	Selected by weighted value-risk-effort objective under constraints.
MESOS-2334	Tasks get stuck in TASK_STAGING after a network decode error.	1	6.98	0.54	Backend	Y	Selected by weighted value-risk-effort objective under constraints.
MESOS-3461	Update Python test framework support for quiesce/revive offer calls.	1	6.83	0.48	Testing	Y	Selected by weighted value-risk-effort objective under constraints.
XD-1	HDFS ItemWriter.	1	5.98	0.54	Database	N	Rejected due to sprint capacity after higher-priority feasible picks.
XD-2	HDFS core writing helper classes.	1	5.90	0.52	Database	N	Rejected due to sprint capacity after higher-priority feasible picks.

modify-and-rerun workflow so that teams can negotiate the final trade-off rather than accept a statically imposed policy.

c) Validity Considerations.: The reported evidence warrants careful interpretation. Internal validity is constrained by reliance on frozen historical CSV snapshots rather than live industrial planning sessions. Construct validity depends on the fidelity of the derived business-value and risk proxies. The inclusion of TAWOS, Spring XD, Usergrid, and Aurora in the evaluation layout is the primary mechanism for addressing external validity. Reproducibility validity remains a concern because local reruns are susceptible to variability in dependency completeness, database initialization, and operating-system permissions.

VI. CONCLUSION

This work has introduced **ApexS**, an explainable, context-aware sprint planning system whose principal contribution is *integration*: four previously separate capabilities—team-specific weight learning, constraint-enforced MILP selection, bidirectional story-level explanation generation, and end-to-end workflow support from dataset ingestion to Jira-compatible export—are unified within a single, deployable Docker Compose stack. Historical sprint data is converted into context; context drives adaptive weight inference; those weights govern binary story selection under binding operational constraints; and the resulting decisions are rendered transparent through explanation, approval, and export mechanisms, all accessible without leaving the application.

Repository-grounded evidence confirms that the system is both usable and analytically meaningful. Across the evaluation snapshot, the learning stage achieves nontrivial signal under held-out evaluation (MAE measured on a test split, with R^2 reported when target variance is non-zero); representative planning scenarios respond correctly to capacity, risk, and skill perturbations (Table VI); learned MILP outperforms greedy and random feasible baselines on all four datasets and improves delivered value versus fixed-weight and context-only MILP (Table VIII); and explanation coverage is complete at 100% across the evaluated runs, with qualitative story-level evidence captured in Table IX. Backend-level feasibility and score-distribution diagnostics further improve traceability by quantifying how many candidates are filtered per constraint class before optimization and how scoring mass is distributed across feasible stories. The system therefore constitutes a credible foundation for further research and for a publication-oriented systems contribution.

Three directions remain open for future investigation. First, *cross-dataset visualization completion*: populating the full chart set for Spring XD, Usergrid, Aurora, and TAWOS would substantially reinforce the empirical presentation. Second, *user study*: establishing whether Scrum teams trust, correctly interpret, and actively use the generated explanations through a controlled experiment is necessary to advance from a systems contribution to a human-factors finding. Third, *infrastructure hardening*: pinning all optional dependencies, completing the migration from SQLite to PostgreSQL, and designating Celery as the exclusive asynchronous execution path would eliminate the reproducibility risks identified in the current evaluation. Because the core architecture is already in place, the path forward calls for broader empirical validation rather than fundamental redesign.

REFERENCES

[1] Y. K. Kula, A. Deshpande, and M. Gharehyazie, "Context-Aware Automated Sprint Plan Generation for Agile Software Development," in *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024. doi: 10.1145/3691620.3695540.

[2] F. Chaves, L. Teixeira, and J. Araujo, "A Lagrangian Heuristic for Sprint Planning in Agile Software Development," in *Proc. SBES 2014*, pp. 1–10, 2014.

[3] A. Ozcelikkan, B. M. Demirors, and A. Tarhan, "A Multi-Objective Agile Project Planning Model and a Comparative Meta-Heuristic Approach," *Information and Software Technology*, vol. 151, art. 107023, 2022. doi: 10.1016/j.infsof.2022.107023.

[4] I. Tawosi, T. K. Sathe, and M. T. Fischer, "A Versatile Dataset of Agile Open Source Software Projects," in *Proc. 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 707–711. doi: 10.1145/3524842.3528029.

[5] L. Montgomery, S. Hess, G. Williams, and A. Meneely, "An Alternative Issue Tracking Dataset of Public Jira Repositories," in *Proc. 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 73–77. doi: 10.1145/3524842.3528486.

[6] M. Perkusich *et al.*, "Intelligent Software Engineering in the Context of Agile Software Development: A Systematic Literature Review," *Information and Software Technology*, vol. 119, art. 106241, 2020. doi: 10.1016/j.infsof.2019.106241.

[7] M. Biesialska, K. Franch, and V. Muntes-Mulero, "Big Data Analytics in Agile Software Development: A Systematic Mapping Study," *Information and Software Technology*, vol. 132, art. 106448, 2021. doi: 10.1016/j.infsof.2020.106448.

[8] M. Mohammadkhani *et al.*, "A Systematic Literature Review of Explainable AI for Software Engineering," *arXiv preprint arXiv:2302.06065*, 2023. doi: 10.48550/arXiv.2302.06065.

[9] G. Yang, D. Ye, and Z. Xia, "A Survey on Explainable AI: From Approaches, Limitations and Applications Aspects," *Human-Centric Intelligent Systems*, vol. 3, pp. 161–188, 2023. doi: 10.1007/s44230-023-00038-y.

[10] P. Hamm *et al.*, "Explanation Matters: An Experimental Study on Explainable AI," *Electronic Markets*, vol. 33, art. 48, 2023. doi: 10.1007/s12525-023-00640-9.

[11] W. Fu and C. Tantithamthavorn, "GPT2SP: A Transformer-Based Agile Story Point Estimation Approach," *IEEE Transactions on Software Engineering*, vol. 49, no. 6, pp. 2723–2738, 2023. doi: 10.1109/TSE.2022.3158252.

[12] E. Ahmad and F.-Y. Kuo, "Improving Story Points Estimation Using Ensemble Machine Learning," *Software Quality Journal*, 2025. doi: 10.1007/s11219-025-09731-6.

[13] N. Pavlic, T. Beranic, and G. Jezic, "Can Large-Language Models Replace Humans in Agile Effort Estimation? Lessons from a Controlled Experiment," *Applied Sciences*, vol. 14, no. 24, art. 12006, 2024. doi: 10.3390/app142412006.

[14] L. M. Rivera Ibarra *et al.*, "Early Estimation in Agile Software Development Projects: A Systematic Mapping Study," *Informatics*, vol. 11, no. 4, art. 81, 2024. doi: 10.3390/informatics11040081.

[15] X. Xiao *et al.*, "Multi-Product Bug Report Assignment via Multi-Task Learning," in *Proc. 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 10–21. doi: 10.1109/ASE.2019.00010.